

“Student Record Management System”

This project was developed using the Python programming language, Tkinter for the graphical user interface, and SQLite for database management.

Project Overview

The main purpose of this system is to help manage student records efficiently.

The system allows users to:

- Add student records
- Update student information
- Delete records
- Search for students
- Manage registration queues
- Undo operations
- Login and register users securely

The project also demonstrates the implementation of three important data structures.

Technologies Used

The technologies used in this project are:

- Python
 - Tkinter GUI
 - SQLite Database
 - JSON for user login storage
-

Data Structures Used

This project implements three data structures:

1. Stack

The first data structure is a Stack.

The stack follows the LIFO principle, which means “Last In, First Out”.

In this system, the stack is used for the Undo feature.

Whenever a user adds, updates, or deletes a student, the action is pushed into the stack.

When the Undo button is clicked, the last action performed is reversed.

Example:

- If a student is deleted,
 - Undo restores that student.
-

2. Queue

The second data structure is a Queue.

The queue follows the FIFO principle, meaning “First In, First Out”.

In this system, the queue is used to manage student registration processing.

Students are added to the queue using the “Add to Queue” button.

The “Process Queue” button processes students in the exact order they entered the queue.

3. Linked List

The third data structure is a Linked List.

The linked list stores student records using nodes connected together.

Each node contains:

- Student data
- A pointer to the next node

The linked list is used for:

- Traversing student records
- Searching records
- Displaying linked list contents

This demonstrates dynamic data storage and traversal.

Database Functionality

The system uses SQLite as the database.

The database stores:

- Student ID
- Student Name
- Program
- Year

The system performs CRUD operations:

- Create
- Read
- Update
- Delete

All student information is saved permanently in the database.

Login System

The project also includes a login and registration system.

Users can:

- Create accounts
- Login securely
- Access the system after authentication

User credentials are stored in a JSON file.

User Interface

The graphical interface was built using Tkinter.

The interface contains:

- Input fields
- Action buttons
- Queue management section
- Student database display

The interface was designed to be simple, clean, and user friendly.

Demonstration

Now I will demonstrate how the system works.

Demonstration Steps:

1. Login into the system
 2. Add a student
 3. Search for a student
 4. Update student information
 5. Delete a student
 6. Undo the delete action
 7. Add students to queue
 8. Process queue
 9. Show linked list contents
-

Challenges Faced

Some of the challenges faced included:

- Connecting the GUI with the database
- Implementing Undo functionality using Stack
- Synchronizing the Linked List with database records
- Managing queue operations correctly

These challenges helped improve understanding of data structures and Python programming.

Conclusion

In conclusion, this project successfully demonstrates:

- Database management
- GUI programming
- User authentication
- Implementation of Stack, Queue, and Linked List data structures

The system improves efficiency in managing student records and provides practical use of data structures in real-world applications.